ORF307
Network optimization

Ioannis Akrotirianakis

Today's topics

- Network flows
- Minimum cost network flow problem
- Network flow solutions
- Examples: maximum flow, shortest path, assignment problem

- **Recap**

Primal and Dual basic feasible solutions

Optimality conditions of the standard form LP

Primal:

$\min\ c^T x$
s.t. $Ax = b$
$x \geq 0$

Dual:

$max\ -b^T y$
s.t. $A^T y + c \geq 0$

Primal feasibility: $A\boldsymbol{x} = b, \boldsymbol{x} \geq 0 \ \Rightarrow\ \boldsymbol{x_B} = A_B^{-1} b \geq 0, \quad \boldsymbol{x_N} = 0 \ \Rightarrow\ \quad \boldsymbol{x} \geq 0$

Dual feasibility: $A^T y + c = 0$. (by construction) Set $\boldsymbol{y} = -A_B^{-T} c_B$. If $\bar{c} = c + A^T y \geq 0$ (reduced cost) then y is dual feasible

Let's see if $\boldsymbol{x_B} = A_B^{-1} b$ and $\boldsymbol{y} = -A_B^{-T} c_B$ are **optimal?**

Duality gap is zero: $c^T \boldsymbol{x} + b^T \boldsymbol{y} = c_B^T x_B + b^T(-A_B^T c_B) = c_B^T x_B - c_B^T A_B^{-1} b = c_B^T x_B - c_B^T x_B = 0$ (by construction)

Optimal value function

$$p^*(u) = \min\{c^T x \mid Ax = b + u, \qquad x \geq 0\}$$

Assumption: $p^*(0)$ is finite

Properties:

- $p^*(u) > \infty$ everywhere (from the global lower bound)
- $p^*(u)$ is piecewise-linear on its domain

Local sensitivity analysis

u is now in a neighborhood of the origin

Original LP:

$$\begin{aligned} \min \quad & c^T x \\ \text{s.t.} \quad & Ax = b \\ & x \geq 0 \end{aligned}$$

Optimal solution:

- Primal: $x_B^* = A_B^{-1} b$ and $x_i^* = -0, \ i \notin B$
- Dual: $y^* = -A_B^{-T} c_B$

Modified LP

$$\begin{aligned} \min \quad & c^T x \\ \text{s.t.} \quad & Ax = b + u \\ & x \geq 0 \end{aligned}$$

Modified Dual

$$\begin{aligned} \max \quad & -(b+u)^T y \\ \text{s.t.} \quad & A^T y + c \geq 0 \end{aligned}$$

Modified optimal solution:

$$x_B^*(u) = A_B^{-1}(b+u) = x_B^* + A_B^{-1} u$$

$$y^*(u) = y^*$$

The optimal basis does not change

Derivative of the optimal value function

Modified optimal solution:

$$x_B^*(u) = A_B^{-1}(b+u) = x_B^* + A_B^{-1}u$$

$$y^*(u) = y^*$$

Optimal value function:

$$p^*(u) = c^T x^*(u) = c^T x + c_B^T A_B^{-1} u$$

$$= p^*(0) - {y^*}^T u \quad \rightarrow \quad (\textbf{affine} \text{ for small } u)$$

Local derivative:

$$\nabla p^*(u) = -y^* \quad \rightarrow \quad (y^* \text{ are called } \textbf{shadow prices})$$

- **Network flows**

Networks

Networks are prevalent in every aspect of our lives

Examples:

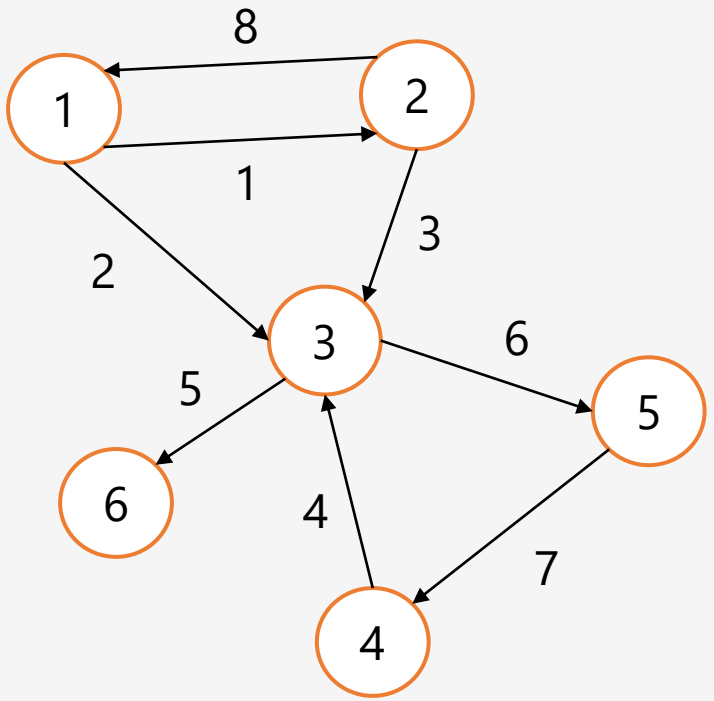
- Electrical and power networks
- Traffic networks
- Supply chain and logistics networks
- Social networks and recommendation systems
- Airline route scheduling
- Telecommunication and data networks
- Blood flow modeling
- Crime and fraud detection
- …

Network modeling

Definition: A **network** (or a directed graph, or digraph) is a **set of m nodes and n directed arcs**.

- Arcs are ordered pairs of nodes (i, j), where i is the **outgoing node** and j the **incoming node**.

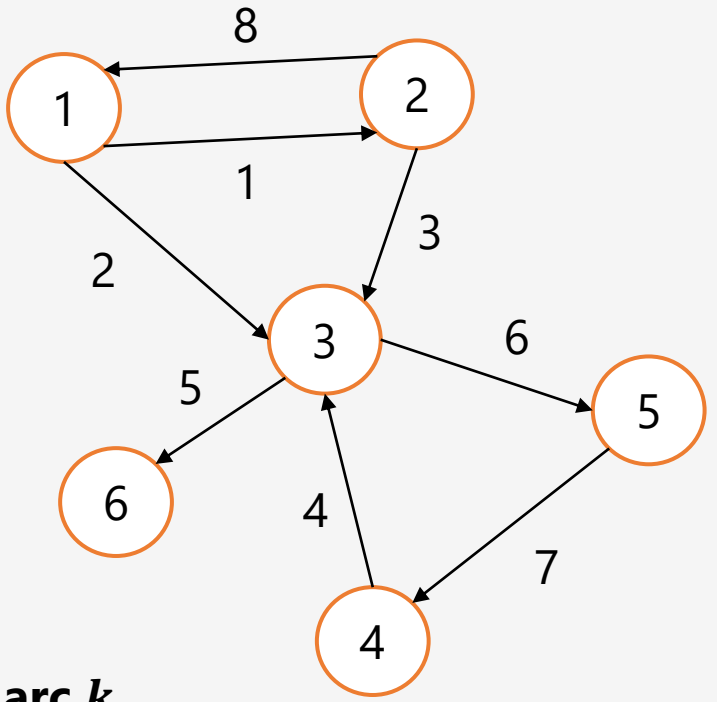
- **Assumptions**:
  - There is **at most one arc** from node i to node j.
  - There are **no self-loops** (i.e., arcs from i to i)



Arc-node incidence matrix

Definition: a matrix $A \in R^{m \times n}$ is called the **incidence matrix** of a network if

$$A_{ik} = \begin{cases} 1, & \text{If arc } k \quad \textbf{starts} \quad \text{at node } i \\ -1, & \text{If arc } k \quad \textbf{ends} \quad \text{at node } i \\ 0, & \text{otherwise} \end{cases}$$



Note: each column has one -1 and one 1

arc k

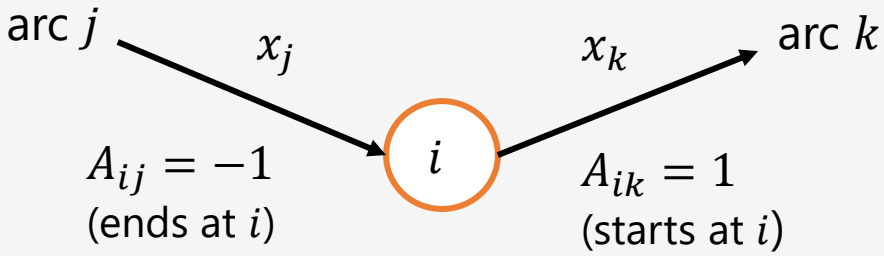
$$A = \begin{bmatrix} 1 & 1 & 0 & 0 & 0 & 0 & 0 & -1 \\ -1 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & -1 & -1 & -1 & 1 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & -1 & 0 \\ 0 & 0 & 0 & 0 & 0 & -1 & 1 & 0 \\ 0 & 0 & 0 & 0 & -1 & 0 & 0 & 0 \end{bmatrix}$$

node i

Network flow

Flow vector $x \in R^n$

- Its j-th element $\boldsymbol{x_j}$ represent the **amount that flows** through arc j
- x_j may represent: material, traffic, information, electricity, etc.



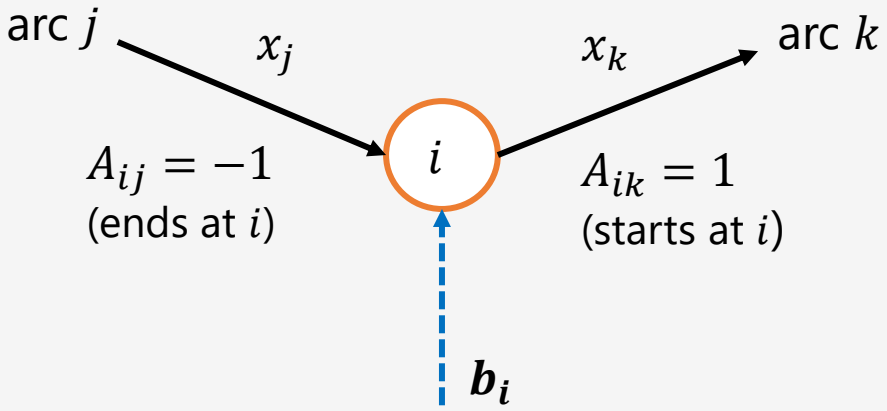
Total flow leaving node i

$$\sum_{k=1}^{n} A_{ik} x_k = (Ax)_i$$

External supply

Supply vector $b \in R^m$:

- b_i is the **external supply** at node i
  - If $\boldsymbol{b_i < 0}$, it represents **demand** at node i
- We must have $\boldsymbol{1^T b = 0}$
  - Total supply = Total demand



Balance equations

$$\sum_{j=1}^{n} A_{ij}x_j = (Ax)_i = b_i, \ \forall i \quad \Rightarrow \quad Ax = b$$

Total outgoing flow at node i

Supply at node i

- **Minimum cost network flow**

Minimum cost network flow problem

General formulation

$$\min c^T x \qquad c, x, u \in R^n$$

$$\text{s.t. } Ax = b \qquad b \in R^m$$

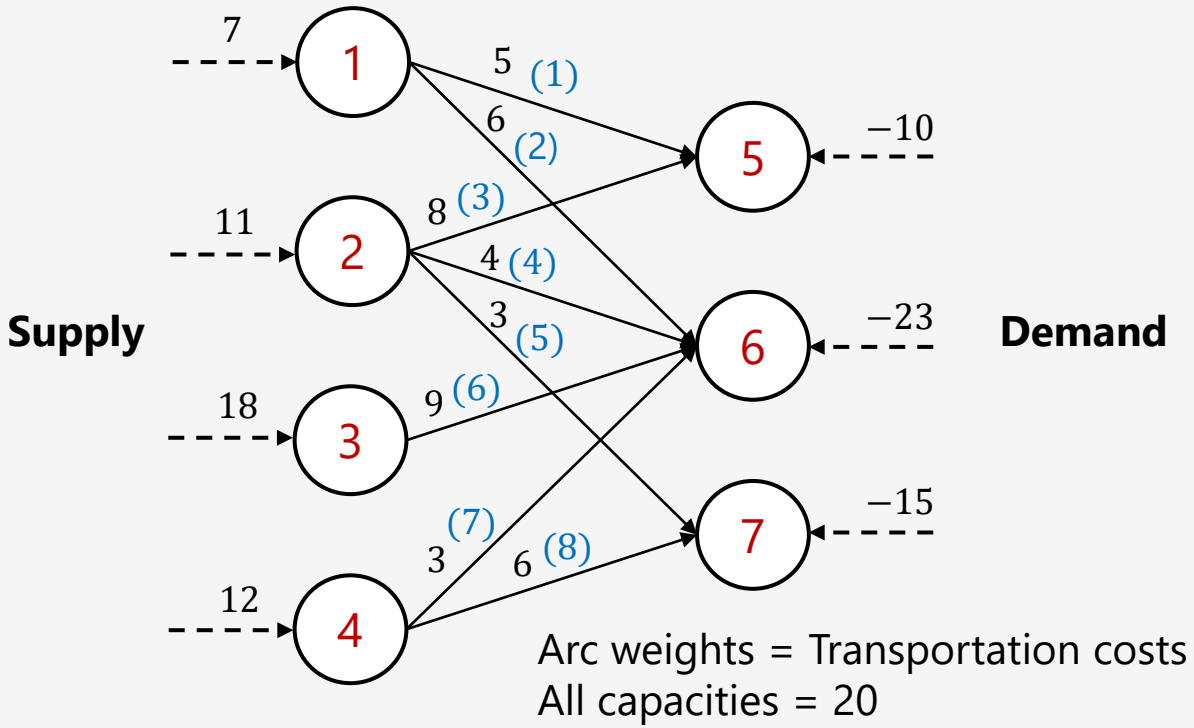
$$0 \leq x \leq u \qquad A \in R^{m \times n}$$

- c_i → the **unit cost of flow** through arc i
- x_i → flow on arc i must be **nonnegative**
- u_i → the **maximum flow capacity** of arc i

Many network optimization problems are a special case of this model

Example - Transportation

Goal: compute the **optimal amount** $x \in R^n$ to be **transported** through the network so that the **demand is satisfied**



Transportation cost: $c = (5,6,8,4,3,9,3,6)^T$

Supply/Demand: $b = (7,11,18,12,-10,-23,-15)^T$

Max. arc capacity: $u = (20,20,20,20,20,20,20,20)^T$

$$A = \begin{array}{c} \\ 1 \\ 2 \\ 3 \\ 4 \\ 5 \\ 6 \\ 7 \end{array} \begin{array}{c} \begin{array}{cccccccc} 1 & 2 & 3 & 4 & 5 & 6 & 7 & 8 \end{array} \\ \begin{bmatrix} 1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 1 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 1 \\ -1 & 0 & -1 & 0 & 0 & 0 & 0 & 0 \\ 0 & -1 & 0 & -1 & 0 & -1 & -1 & 0 \\ 0 & 0 & 0 & 0 & -1 & 0 & 0 & -1 \end{bmatrix} \end{array}$$

Minimum cost network flow:

$$\begin{aligned} \min\ & c^T x \\ s.t.\ \ & Ax = b \\ & 0 \le x \le u \end{aligned}$$

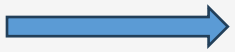
- **Network flow solutions**

Remove arc capacities

Goal: create equivalent network **without arc capacities**

Minimum cost network flow:

$$\min\ c^T x$$
$$s.t.\ \ Ax = b$$
$$0 \le x \le u$$



Minimum cost network flow (standard form *LP*):

$$\min\ \tilde{c}^T x$$
$$s.t.\ \ \tilde{A}\tilde{x} = \tilde{b}$$
$$\tilde{x} \ge 0$$

Remove arc capacities

Idea: use slack variables

$$x_j \le u_j \quad \Rightarrow \quad x_j + s_j = u_j \,,\, s_j \ge 0$$

But the network structure is **lost**

(no longer a "+1" and a "−1" per column)

$$\ldots + x_j \ldots = b_p$$

$$\ldots - x_j \ldots = b_q$$

$$x_j + s_j \ldots = u_j \Rightarrow$$

solve for x_j

$$\Rightarrow x_j \ldots = u_j - s_j$$

Substitute to p-th constraint

$$\ldots - s_j = b_p - u_j$$

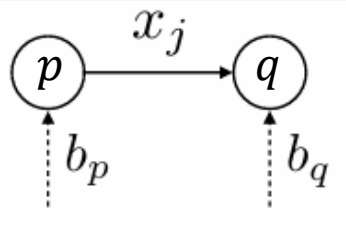
$$\ldots - x_j + \cdots = b_q$$

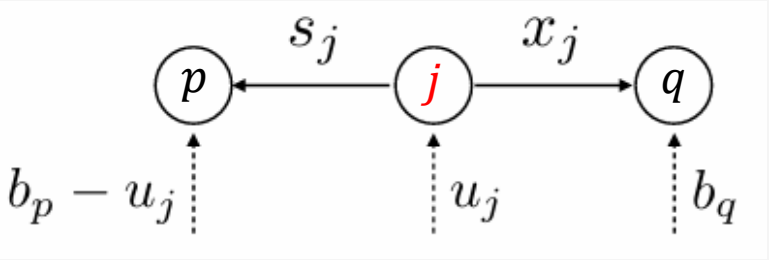
$$x_j \ldots + s_j = u_j$$

Network structure **recovered**

(new node and new arc)

Nodes/arcs interpretation

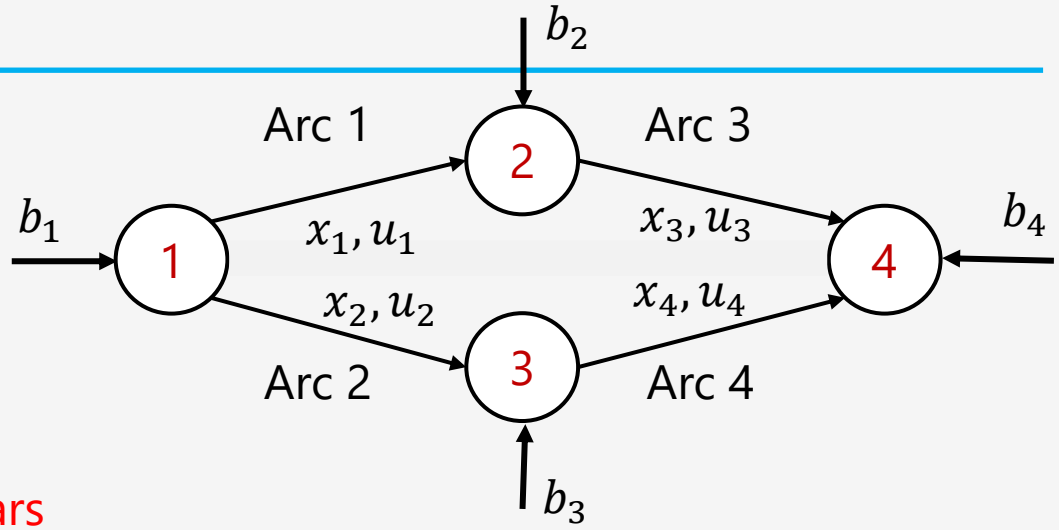




Small example for illustration

$$
\begin{array}{llllll}
+1x_1 & +1x_2 & +0x_3 & +0x_4 & = & b_1 \\
-1x_1 & 0x_2 & +1x_3 & 0x_4 & = & b_2 \\
0x_1 & -1x_2 & 0x_3 & +1x_4 & = & b_3 \\
0x_1 & 0x_2 & -1x_3 & -1x_4 & = & b_4 \\
x_1 & & & & \leq & u_1 \\
 & x_2 & & & \leq & u_2 \\
 & & x_3 & & \leq & u_3 \\
 & & & x_4 & \leq & u_4
\end{array}
$$

$Ax = b$
$x \leq u$ $\Rightarrow$



Adding slack vars

$\Downarrow$

$$
\begin{array}{llllllll|ll}
+1x_1 & +1x_2 & +0x_3 & +0x_4 & +0s_1 & +0s_2 & +0s_3 & +0s_4 & = & b_1 \\
-1x_1 & 0x_2 & +1x_3 & 0x_4 & & & & & = & b_2 \\
0x_1 & -1x_2 & 0x_3 & +1x_4 & & & & & = & b_3 \\
0x_1 & 0x_2 & -1x_3 & -1x_4 & & & & & = & b_4 \\
x_1 & & & & +s_1 & & & & = & u_1 \\
 & x_2 & & & & +s_2 & & & = & u_2 \\
 & & x_3 & & & & +s_3 & & = & u_3 \\
 & & & x_4 & & & & +s_4 & = & u_4
\end{array}
\quad \Rightarrow \quad
\begin{array}{l}
x_1 = u_1 - s_1 \\
x_2 = u_2 - s_2 \\
x_3 = u_3 - s_3 \\
x_4 = u_4 - s_4
\end{array}
$$

Solving for x_i and then substitute to equations where x_1, x_2, x_3, x_4 have a **positive sign**

$\Rightarrow$

$$
\begin{array}{cccccccccccc}
 & 0x_1 & 0x_2 & +0x_3 & +0x_4 & -1s_1 & -1s_2 & +0s_3 & +0s_4 & = & b_1 - u_1 - u_2 & \\
 & -1x_1 & 0x_2 & 0x_3 & 0x_4 & & & -1s_3 & & = & b_2 - u_3 & \\
 & 0x_1 & -1x_2 & 0x_3 & 0x_4 & & & & -1s_4 & = & b_3 - u_4 & \\
 & 0x_1 & 0x_2 & -1x_3 & -1x_4 & & & & & = & b_4 & \\
\Rightarrow & x_1 & & & & +s_1 & & & & = & u_1 & \Rightarrow \\
 & & x_2 & & & & +s_2 & & & = & u_2 & \\
 & & & x_3 & & & & +s_3 & & = & u_3 & \\
 & & & & x_4 & & & & +s_4 & = & u_4 &
\end{array}
$$

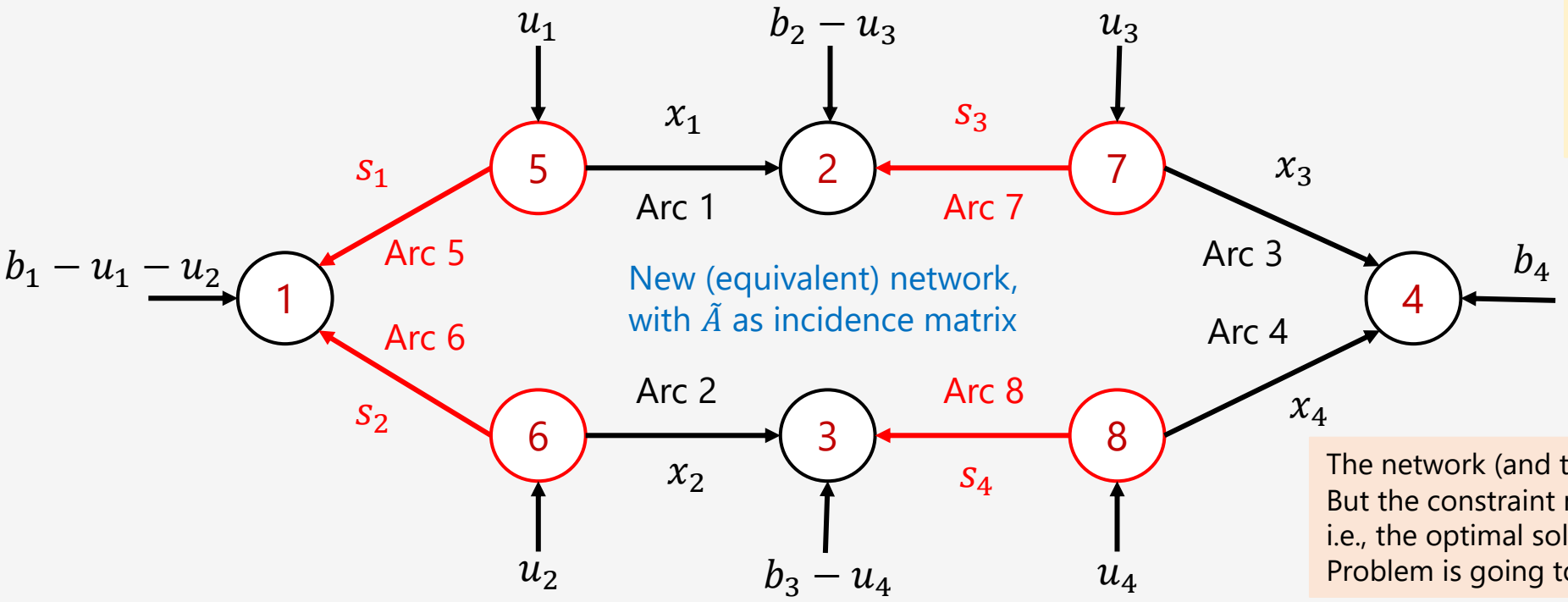
All columns have one $+1$ and one -1 now…

$$\Rightarrow \begin{bmatrix} 0 & 0 & 0 & 0 & -1 & -1 & 0 & 0 \\ -1 & 0 & 0 & 0 & 0 & 0 & -1 & 0 \\ 0 & -1 & 0 & 0 & 0 & 0 & 0 & -1 \\ 0 & 0 & -1 & -1 & 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \\ s_1 \\ s_2 \\ s_3 \\ s_4 \end{bmatrix} = \begin{bmatrix} b_1 - u_1 - u_2 \\ b_2 - u_3 \\ b_3 - u_4 \\ b_4 \\ u_1 \\ u_2 \\ u_3 \\ u_4 \end{bmatrix}$$

$\Rightarrow \widetilde{\boldsymbol{A}}\widetilde{\boldsymbol{x}} = \widetilde{\boldsymbol{b}}$ ($\widetilde{\boldsymbol{A}}$ is Totally Unimodular)

$$\min\ \tilde{c}^T\,\tilde{x}$$
$$s.t.\ \ \widetilde{\boldsymbol{A}}\,\widetilde{\boldsymbol{x}} = \widetilde{\boldsymbol{b}}$$
$$\tilde{x} \geq 0$$

Min Cost Network Flow in standard LP form



The network (and the size of the LP) have increased
But the constraint matrix $\tilde{A}$ is **Totally Unimodular**,
i.e., the optimal solution of the new Min Cost Flow
Problem is going to contain **integer** values ($\tilde{x}_i \in Z, \forall i$)

Small example for illustration

$$\underbrace{\begin{bmatrix} 0 & 0 & 0 & 0 & -1 & -1 & 0 & 0 \\ -1 & 0 & 0 & 0 & 0 & 0 & -1 & 0 \\ 0 & -1 & 0 & 0 & 0 & 0 & 0 & -1 \\ 0 & 0 & -1 & -1 & 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 & 1 \end{bmatrix}}_{\tilde{A}} \underbrace{\begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \\ s_1 \\ s_2 \\ s_3 \\ s_4 \end{bmatrix}}_{\tilde{x}} = \underbrace{\begin{bmatrix} b_1 - u_1 - u_2 \\ b_2 - u_3 \\ b_3 - u_4 \\ b_4 \\ u_1 \\ u_2 \\ u_3 \\ u_4 \end{bmatrix}}_{\tilde{b}}$$

$\tilde{A}$ is Totally Unimodular matrix

Minimum cost network flow (standard form *LP*):

$$\min \ \tilde{c}^T x$$

$$s.t. \ \ \tilde{A}\tilde{x} = \tilde{b}$$

$$\tilde{x} \geq 0$$

Equivalent un-capacitated network

Minimum cost network flow (standard form LP):

$$\min \ \tilde{c}^T x$$
$$s.t. \ \ \tilde{A}\tilde{x} = \tilde{b}$$
$$\tilde{x} \geq 0$$

- Matrix A is **still** an arc-node incidence matrix
- You can apply **Duality Theory** and **Sensitivity Analysis** straightforwardly (LP is is standard form)

- But can we say something about the **extreme points??**

Total unimodularity

- **Definition**: a **minor** of a matrix $A \in R^{m \times n}$ is the determinant of a square submatrix of A.
- **Definition**: A matrix $A \in R^{m \times n}$ is **totally unimodular** if **all its minors** are $-1, 0,$ or 1.

  - **Example**: a **node-arc incidence matrix** of a directed graph is **totally unimodular**

$$A = \begin{bmatrix} 1 & 1 & 0 & 0 & 0 & 0 & 0 & -1 \\ -1 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & -1 & -1 & -1 & 1 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & -1 & 0 \\ 0 & 0 & 0 & 0 & 0 & -1 & 1 & 0 \\ 0 & 0 & 0 & 0 & -1 & 0 & 0 & 0 \end{bmatrix}$$

Properties:

- The **entries** A_{ij} of A have values: $-1, 0,$ or 1.
- The **inverse** of any nonsingular square **submatrix** of A has entries $-1, 0,$ or 1.

Integrality theorem

Theorem: Consider a polyhedron $P = \{x \in R^n \mid Ax = b,\ x \geq 0\}$, where $A \in R^{m \times n}$ is a totally unimodular matrix and $b \in R^m$ is an integer vector. Then **all extreme points** of P are **integer vectors**.

Proof: recall that all **extreme points** are **basic feasible solutions** with

$$x_B = A_B^{-1} b \text{ and } x_i = 0, \forall i \notin B.$$

Also, A_B^{-1} has integer elements because A is totally unimodular matrix

Note also that $\boldsymbol{b}$ **has integer elements**

Hence the vector $x_B = A_B^{-1} b$ must have integer elements, and so does x. ■

Implications for network and combinatorial optimization

Minimum cost network flow

$$\min\ c^T x$$
$$s.t.\ \ Ax = b$$
$$0 \le x \le u$$

→ **If** b and u are integral vectors, then the optimal solution x^* is integral too

Integer Linear Program (ILP)

$$\min\ c^T x$$
$$s.t.\ \ Ax = b$$
$$0 \le x \le u$$
$$x \in Z^n$$

→ **ILP**s are very difficult to solve (NP-complete)

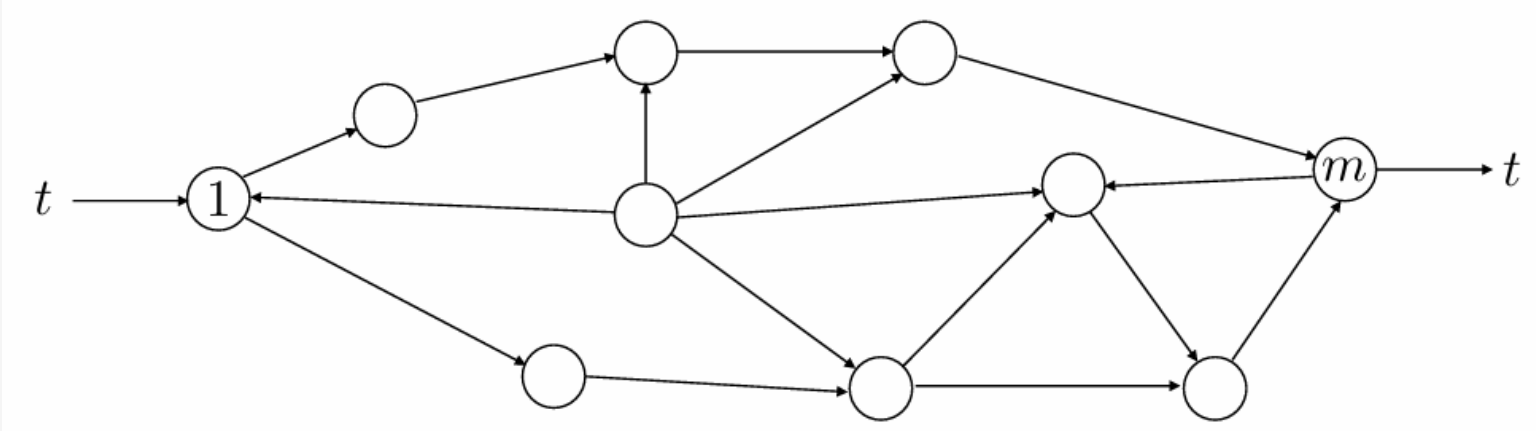
↓

If A is totally unimodular and b, u are integral, then we can **relax** the integrality constraint $x \in Z^n$, **solve the LP instead, and still get an integral solution**.

- **Examples**

Maximum flow problem

Goal: maximize the flow in the network from node 1 (**source**) to node m (**sink**)



$$\max \ t$$

$$s.t. \ \ Ax = te$$

$$0 \le x \le u$$

$$e = \begin{bmatrix} 1 \\ 0 \\ \vdots \\ 0 \\ -1 \end{bmatrix}$$

- Flow must **conserve mass** at each node

$$FlowIN = FlowOUT$$

- Flow must **respect the capacities** u of each arc

x: is the **flow vector** and t: is the **demand** at the sink node

Example:

- A city's **water utility** needs to **route water** from a main reservoir (**source**) to a distribution station (**sink**) through a **network of pipes**.
- Each pipe has a **maximum capacity**, and the **goal** is to **maximize** **the amount of water delivered to the station without exceeding these limits.**

$$\max \ t$$

$$s.t. \ Ax = te$$

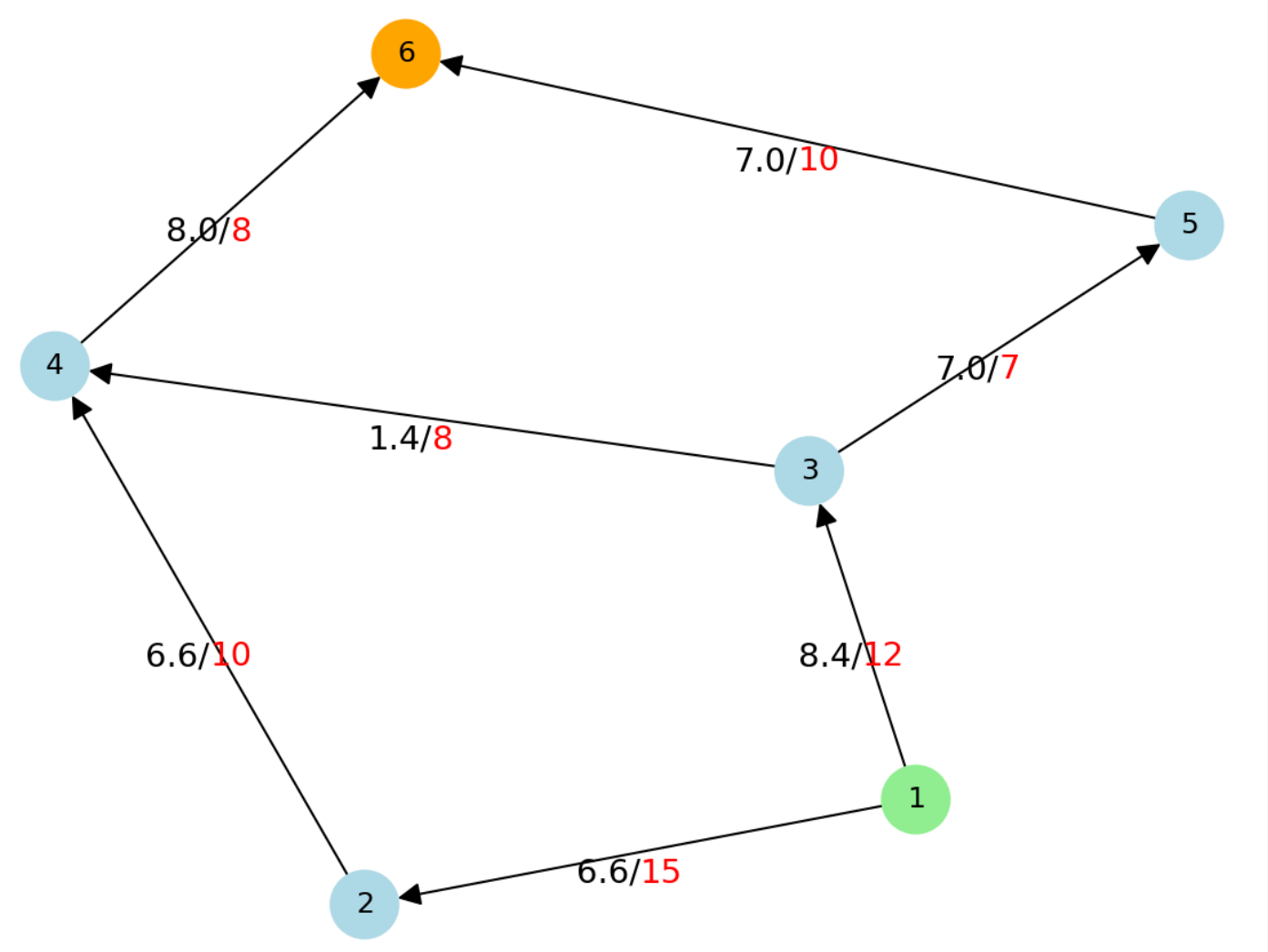
$$0 \le x \le u$$

Network structure:

- Node 1 is the source node
- Node 6 is the sink node (distribution node)
- Nodes 2,3,4,5 are junction nodes.

$$A = \begin{bmatrix} 1 & 1 & 0 & 0 & 0 & 0 & 0 \\ -1 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & -1 & 0 & 1 & 1 & 0 & 0 \\ 0 & 0 & -1 & -1 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & -1 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 & -1 & -1 \end{bmatrix} \quad e = \begin{bmatrix} +1 \\ 0 \\ 0 \\ 0 \\ 0 \\ -1 \end{bmatrix} \quad u = \begin{bmatrix} 15 \\ 12 \\ 10 \\ 8 \\ 7 \\ 8 \\ 10 \end{bmatrix}$$

Example

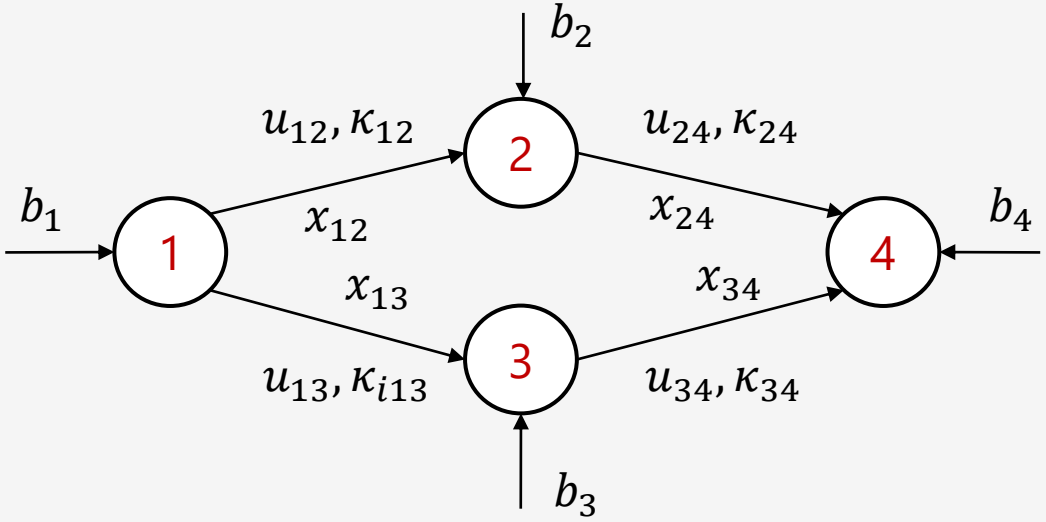


Capacities in red

Optimal flow in black

The Minimum Cost Flow (MCF) problem

- It is a **generalization** of the Max Flow Problem that introduces two new important concepts
  - κ_{ij} : the **cost per unit of flow** on each edge (i,j)
  - Each node i has a required **flow balance** b_i
    - **Supply** node: $b_i > 0$ (source)
    - **Demand** node: $b_i < 0$ (sink)
    - **Transshipment** node: $b_i = 0$



- Network is represented as a **graph**: $G = (V, E)$
- **Incoming nodes** to node j : $I(j) = \{i \in V \,|(i,j) \in E\}$
- **Outgoing nodes** to node j : $O(j) = \{\ell \in V \,|(j,\ell) \in E\}$

$$\min \sum_{(i,j)\in E} \kappa_{ij} x_{ij}$$

$$s.t. \sum_{\ell \in O(j)} x_{j\ell} - \sum_{i \in I(j)} x_{ij} = b_j \quad \forall j \in V$$

$$0 \le x_{ij} \le u_{ij} \quad \forall (i,j) \in E$$

$$\min \textstyle\sum_{(i,j)\in E} \kappa_{ij} x_{ij}$$

$$s.t. \quad Ax = b$$

$$0 \le x \le u$$

$$b = \begin{bmatrix} b_1 \\ b_2 \\ \vdots \\ b_{|V|-1} \\ b_{|V|} \end{bmatrix}$$

Max Flow Problem as a Min Cost Flow problem

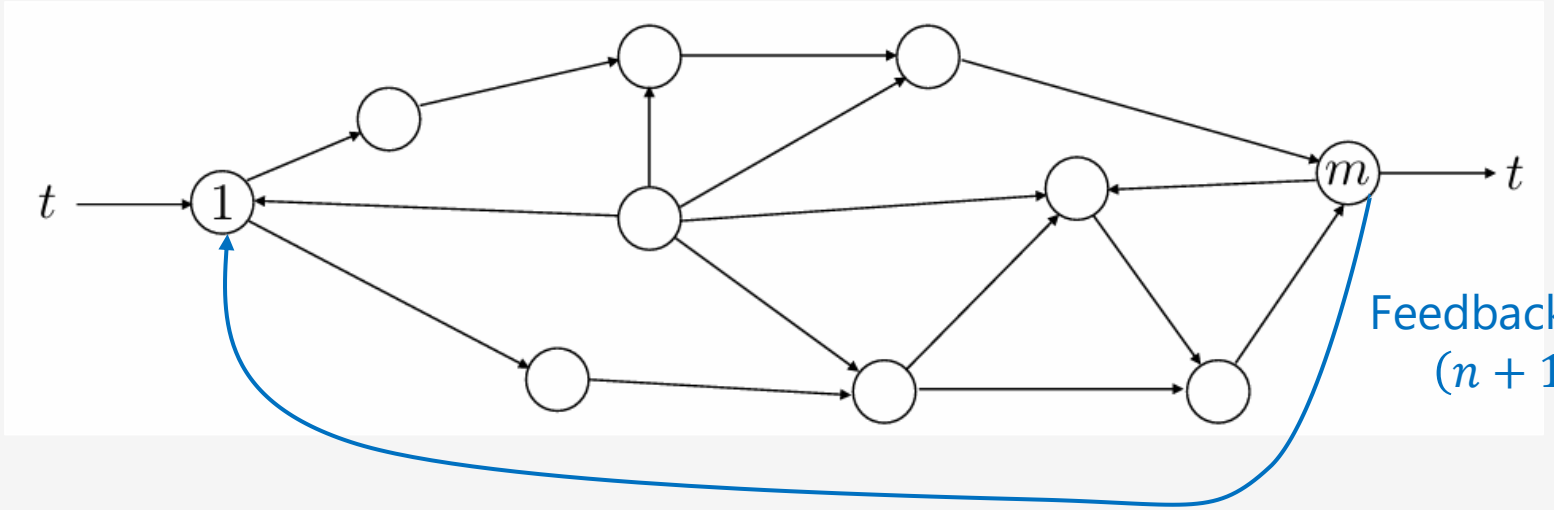
The steps to follow:

- Add a **feedback edge** to the Max Flow problem: from the sink node to the source node.
- Set **all costs** per unit flows to **zero**: $\kappa_{ij} = 0, \forall (i,j) \in E$
- Set the **cost from the sink node to the source node** to -1 : $\kappa_{m1} = -1$
- Set the **capacity from the sink node to the source node** to ∞ : $c_{m1} = \infty$
- Set **balances** to all nodes to **zero** : $b_i = 0 \;\; (FlowIN = FlowOUT)$
- Define the **objective**: **minimize** the negative flow from sink to source: $\min -x_{m1}$

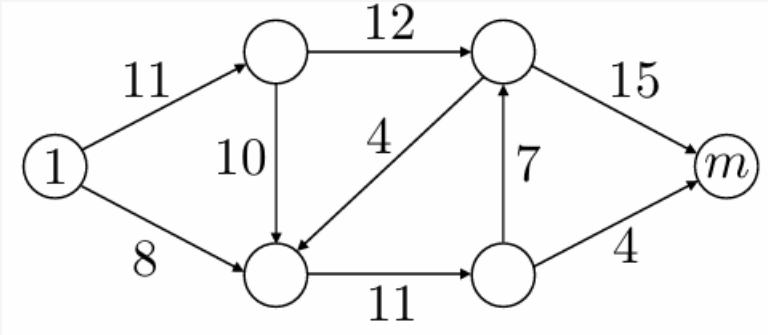
$$\min \;\; -t$$

$$s.t. \;\; [A \quad -e] \begin{bmatrix} x \\ t \end{bmatrix} = 0$$

$$0 \leq \begin{bmatrix} x \\ t \end{bmatrix} \leq \begin{bmatrix} u \\ \infty \end{bmatrix}$$

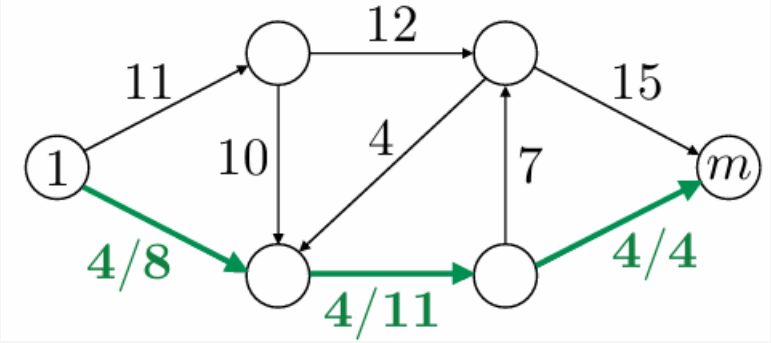


Maximum flow example – different paths

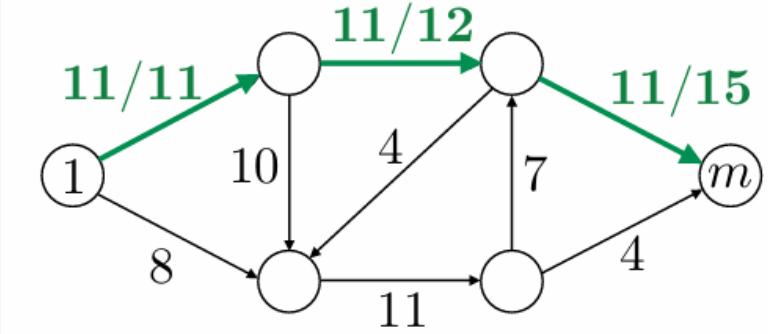


Capacities on the arcs

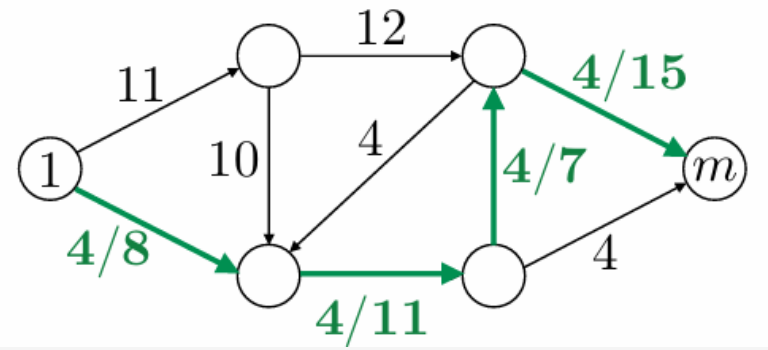
First flow



Second flow



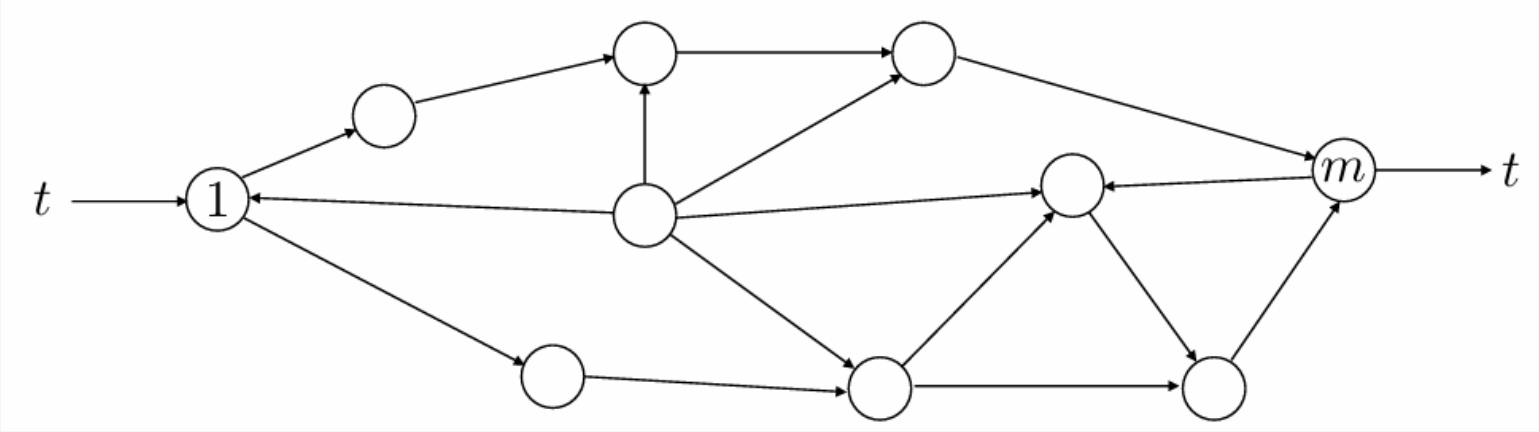
Third flow



Flow/Capacity

Shortest path problem

Goal: find the shortest path between nodes 1 and m



Paths can be represented as $\mathbf{0-1}$ **vectors,** $\boldsymbol{x \in \{0,1\}^n}$

$$\min\ c^T x$$

$$s.t.\ \ Ax = e$$

$$x \in \{0,1\}^n$$

- c_j is the "length" of arc j
- $e = (1,0,\dots,0,-1)^T$
- The variables $\boldsymbol{x_j}$ can only take **0 or 1 values**
  - $x_j = 1$, if arc j is **in** the shortest path
  - $x_j = 0$, otherwise

Shortest path as minimum cost flow

(IP)

$$\min\ c^T x$$
$$s.t.\ \ Ax = e$$
$$x \in \{0,1\}^n$$

A is **unimodular**
e is integer vector

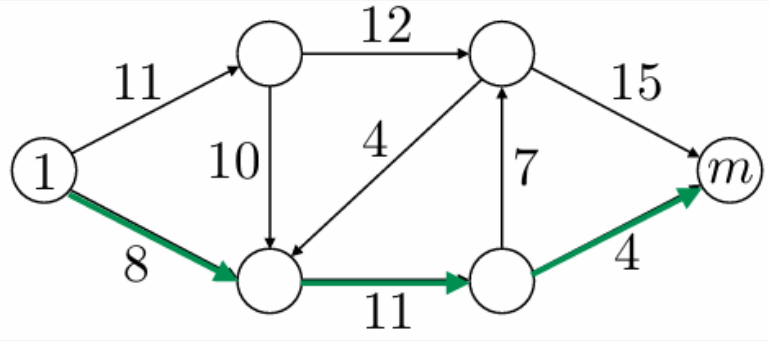
$$\min\ c^T x$$
$$s.t.\ \ Ax = e$$
$$0 \le x \le 1$$

(LP-relaxation)

Easier to solve!

Extreme points satisfy $x_i \in \{0, 1\}$

Example (arc costs shown)



$$c = (11,8,10,12,4,11,7,15,4)^T$$
$$x^* = (0,1,0,0,0,1,0,0,1)^T$$
$$c^T x^* = 24$$

Assignment problem

Goal: match N people to N tasks

- Each **person** assigned to one **task**
- Each **task** assigned to one **person**
- C_{ij}: the **cost of matching** person i to task j
- $X_{ij} = 1$, if person i assigned to task j
- $X_{ij} = 0$, otherwise

IP formulation:

$$\min \sum_{i=1}^{N} \sum_{j=1}^{N} C_{ij} X_{ij}$$

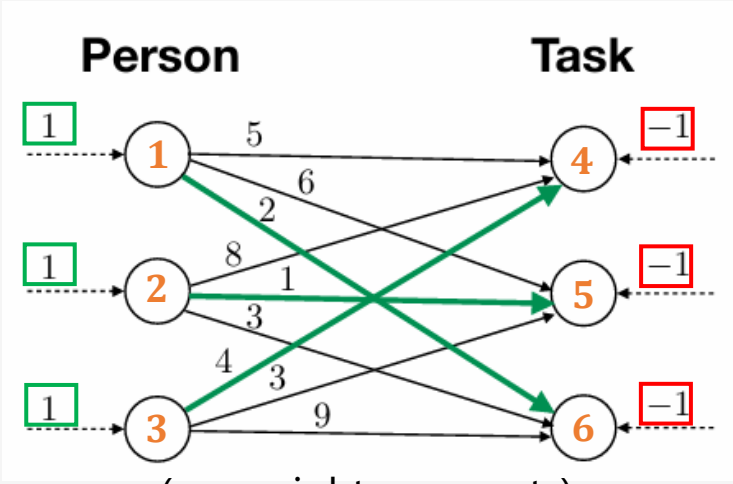
$$\text{s.t.} \sum_{i=1}^{N} X_{ij} = 1 \, , j = 1, \dots, N$$

$$\sum_{j=1}^{N} X_{ij} = 1 \, , i = 1, \dots, N$$

$$X_{ij} \in \{0,1\}, \forall i, j$$

How can we define the network?

Task assignments as minimum cost network flow



(arc weights are costs)

LP formulation→

Minimum cost network flow

$$\min\ c^T x$$

$$s.t.\ \ Ax = b$$

$$0 \le x \le 1$$

Extreme points satisfy $x_i \in \{0,1\}$

$$A = \begin{bmatrix} 1 & 1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 1 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 1 & 1 \\ -1 & 0 & 0 & -1 & 0 & 0 & -1 & 0 & 0 \\ 0 & -1 & 0 & 0 & -1 & 0 & 0 & -1 & 0 \\ 0 & 0 & -1 & 0 & 0 & -1 & 0 & 0 & -1 \end{bmatrix}$$

$$c = (5,6,2,8,1,3,4,3,9)^T, \quad b = (1,1,1,-1,-1,-1)^T$$

Optimal solution:

$$\boldsymbol{x}^* = (0,0,\mathbf{1},0,\mathbf{1},0,0,0,\mathbf{1})^T$$

$$c^T x^* = 7$$

References

- D. Bertsimas & J. Tsitsiklis: Introduction to Linear Optimization
  - Ch. 7: Network flow problems
- R. Vanderbei: Linear Programming: Foundations and Extensions
  - Ch. 14: Network flow problems
  - Ch. 15: Applications

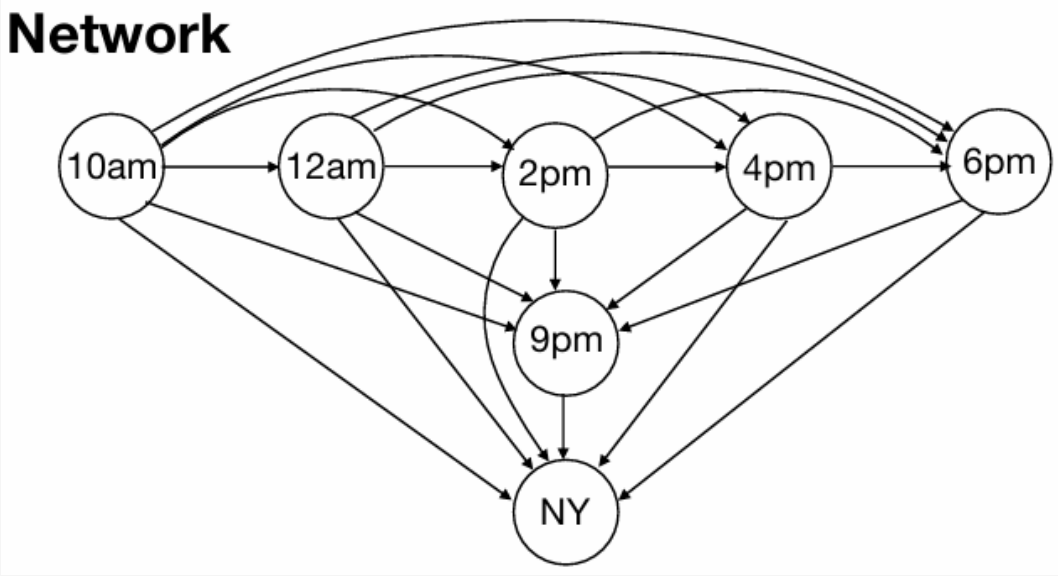
Next lecture

- Interior Point Algorithms

- **Appendix**

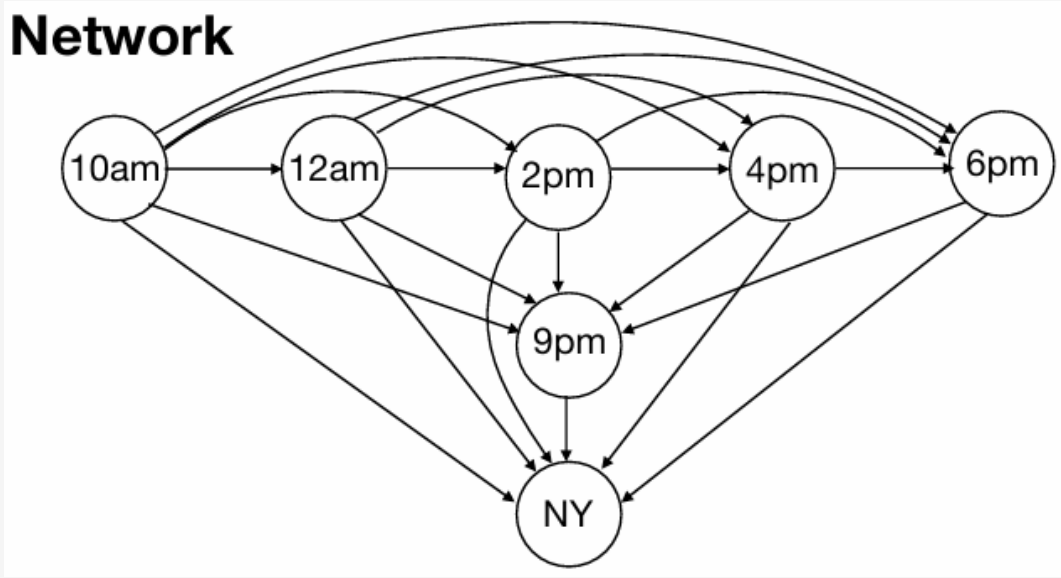
Example – Airline passenger routing

- United Airlines has 5 flights per day from Boston to New York
- Flight capacities: $u = (100,100,100,150,150)^T$
- Costs: \$50/hour of delay
- Last option: 9pm flight with other company (pay additional cost of \$75)
- Today's reservations: $(110,118,103,161,140)^T$



Example – Airline passenger routing

- **Decision variables:**
  - x_j: passengers "flowing" on arc i
- **Costs**:
  - c_j: cost of moving a passenger on arc j
    - Between flights: \$50/hour
    - To 9pm flight: \$75 additional
    - To NY: \$0 (as scheduled)
- **Supplies**:
  - b_i:reserved passengers for flight i
    - 9pm flight: $b_i = 0$
    - NY supply: -total reserved passengers
- **Capacities**:
  - u_j: maximum passengers over arc j
    - Between flights: $u_j = \infty$
    - To NY: u_i =flight capacity



Minimum cost network flow:

$$\min \ c^T x$$

$$s.t. \ \ Ax = b$$

$$0 \leq x \leq u$$